L41 — Heap Sort and Comparative Analysis

Unit: 3 | **Chapter:** Sorting | **BT Level:** BT5 | **CO:** CO3, CO5

Learning Objectives

- Implement heap sort using a max-heap
- Trace heap sort step by step
- Prove O(n log n) time complexity
- Compare heap sort against merge sort and quick sort

---

1. Heap Sort Idea

Heap sort uses a max-heap to sort an array in-place:

1. **Build max-heap** from array — O(n)
2. **Repeatedly extract max:**
    - Swap root (max) with last element
    - Reduce heap size by 1
    - Sift down the new root
    - Repeat n-1 times — O(n log n)

After n-1 extractions, the array is sorted in ascending order.

---

2. Implementation

```
def heap_sort(arr):
    """
    Sort array in-place using heap sort.
    Time: O(n log n)  |  Space: O(1)
    """
    n = len(arr)

    def sift_down(arr, root, size):
        while True:
            largest = root
            left = 2 * root + 1
            right = 2 * root + 2

            if left < size and arr[left] > arr[largest]:
                largest = left
            if right < size and arr[right] > arr[largest]:
                largest = right

            if largest != root:
                arr[root], arr[largest] = arr[largest], arr[root]
```

```
            root = largest
        else:
            break

# Step 1: Build max-heap
for i in range(n // 2 - 1, -1, -1):
    sift_down(arr, i, n)

# Step 2: Extract max n-1 times
for end in range(n - 1, 0, -1):
    arr[0], arr[end] = arr[end], arr[0]   # Move max to sorted posi
    sift_down(arr, 0, end)                # Restore heap for reduce

return arr
```

---

3. Detailed Trace

Array: `[4, 10, 3, 5, 1, 9]`

Step 1: Build Max-Heap

Start from last internal node (index = n//2 - 1 = 2):

i	Sift down	Array
2	sift_down(2): children are 9 (idx 5). 9 > 3 → swap	`[4, 10, 9, 5, 1, 3]`
1	sift_down(1): children 5(idx 3),1(idx 4). 10 > 5 → no swap	`[4, 10, 9, 5, 1, 3]`
0	sift_down(0): children 10(idx1),9(idx2). 10 > 4 → swap	`[10, 4, 9, 5, 1, 3]`
	Continue: 4 at idx 1, children 5(idx3),1(idx4). 5 > 4 → swap	`[10, 5, 9, 4, 1, 3]`

Max-heap built: `[10, 5, 9, 4, 1, 3]`

Step 2: Extraction (heap_size shrinks each time)

Round	Swap root with	Array	Heap portion	Sorted portion
1	arr[0]↔arr[5]: 10↔3	`[3, 5, 9, 4, 1, 10]` → sift [3,5,9,4,1] → `[9,5,3,4,1, 10]`	`[9,5,3,4,1]`	`[10]`
2	arr[0]↔arr[4]: 9↔1	`[1,5,3,4, 9,10]` → sift → `[5,4,3,1, 9,10]`	`[5,4,3,1]`	`[9,10]`
3	arr[0]↔arr[3]: 5↔1	`[1,4,3, 5,9,10]` → sift → `[4,1,3, 5,9,10]`	`[4,1,3]`	`[5,9,10]`
4	arr[0]↔arr[2]: 4↔3	`[3,1, 4,5,9,10]` → sift → `[3,1, 4,5,9,10]`	`[3,1]`	`[4,5,9,10]`
5	arr[0]↔arr[1]: 3↔1	`[1, 3,4,5,9,10]`	done	all sorted

Sorted: **`[1, 3, 4, 5, 9, 10]`** ✓

---

4. Complexity Proof

Build heap: O(n) (shown in L40)

n-1 extractions: Each extraction:

- Swap: O(1)
- Sift down: O(log k) where k = current heap size

Total extraction cost: $$\sum_{k=2}^{n} \log k = \log(n!) = O(n \log n)$$

By Stirling's approximation: $\log(n!) \approx n \log n - n \approx O(n \log n)$

Total: O(n) + O(n log n) = **O(n log n)** for all cases.

---

5. Comparative Analysis

Property	Heap Sort	Merge Sort	Quick Sort	Insertion Sort
Best case	O(n log n)	O(n log n)	O(n log n)	O(n)
Average case	O(n log n)	O(n log n)	O(n log n)	$O(n^2)$
Worst case	O(n log n)	O(n log n)	$O(n^2)$	$O(n^2)$
Space	O(1) in-place	O(n)	O(log n) avg	O(1)
Stable?	No	Yes	No (typical)	Yes
Cache friendly?	Poor	Good	Good	Excellent
Adaptive?	No	No	Partially	Yes
Practical speed	Slower (poor cache)	Fast	Fast	Fast for small n

---

6. Why Heap Sort is Slow in Practice

Despite O(n log n) worst-case, heap sort is often **slower than quicksort** in practice due to **poor cache locality**:

- Heap operations jump between distant array indices (child = $2i+1$, $2i+2$)
- These jumps cause frequent **cache misses** in modern CPUs
- Quick sort has better locality — it accesses contiguous memory regions
- Merge sort uses sequential access patterns (merge step)

Typical performance ranking (in practice):
Quick sort > Merge sort > Heap sort

When to use heap sort:

- When worst-case O(n log n) guarantee is required AND in-place sorting is needed
- Embedded systems with limited memory (O(1) space)
- Real-time systems that need bounded running time

---

7. k Largest / k Smallest Elements

```
import heapq

def k_largest(arr, k):
    """Find k largest elements efficiently using min-heap. O(n log k)"""
    return heapq.nlargest(k, arr)

def k_smallest(arr, k):
    """Find k smallest elements. O(n log k)"""
    return heapq.nsmallest(k, arr)

# Alternative: partial heap sort
def k_largest_manual(arr, k):
    """Build max-heap, extract max k times."""
    heap = arr[:]
    n = len(heap)
    # Build max-heap
    for i in range(n // 2 - 1, -1, -1):
        _sift_down(heap, i, n)

    result = []
    for _ in range(k):
        result.append(heap[0])
        heap[0] = heap[n - 1]
        n -= 1
        _sift_down(heap, 0, n)

    return result
```

Summary

- **Heap sort:** Build max-heap (O(n)), then repeatedly swap root with end and sift down (O(n log n)).
- Time: O(n log n) in **all cases** (best, average, worst) — guaranteed.
- Space: O(1) — in-place sorting.
- Not stable (equal elements may be reordered).
- Slower in practice than quick sort due to poor cache locality.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 6 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 4.1 (Springer, 2020)